

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – DEBUGGING & OPTIMISATION TASK

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO2 – Manipulation (BL1, BL2, BL3)	Assessment:	Assessment Tool 3 – Debugging & Optimisation Task
Weightage:	10% of CIA	Total Marks:	100
CO2 Statement:	Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems (BL1, BL2, BL3)		
Student Name:	M Vishnu Vardhan	Reg. No.:	192472087
Section / Batch:	2024	Date:	29/07/2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given program/code segment before identifying errors.
- Clearly identify logical, syntactical, and performance-related issues in the code.
- Apply systematic debugging techniques to correct the errors effectively.
- Optimize the given solution by improving efficiency, reducing unnecessary computations, or enhancing time complexity wherever possible.
- Provide proper justification for all corrections and optimization decisions.
- Write clean, structured, and well-documented code with appropriate comments.
- Maintain clarity, logical reasoning, and technical accuracy throughout the solution.

Question Type	Focus Area	Questions
Debugging & Optimisation Task	Identify errors, debug programs, and optimize algorithm efficiency using appropriate techniques	Q1

SECTION A – DEBUGGING & OPTIMISATION TASK

Q45. Debugging Closest Pair Code (Redundant Self-Comparison and Wrong Pairing Logic)

The following brute force Closest Pair implementation compares every point with every other point, including itself, which leads to incorrect minimum distance and redundant computation. Carefully analyze the nested loop structure and explain why self-comparison breaks correctness. Debug the algorithm to compare only valid distinct pairs exactly once. Optimize the implementation by reducing duplicate distance evaluations while preserving brute force behavior. Analyze the time complexity of the corrected solution and rewrite the final code with suitable comments.

```
import math
def closest_pair(points):
    min_dist = float('inf')
    for i in range(len(points)):
        for j in range(len(points)):
            dist = math.sqrt((points[i][0]-points[j][0])**2 +
                             (points[i][1]-points[j][1])**2)
            if dist < min_dist:
                min_dist = dist
    return min_dist
```

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, well-structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Identification	20%				
Debugging	25%				
Optimization	20%				
Analysis	20%				
Code Quality	15%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name: _____	Name: _____	Name: _____
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Q45. Debugging Closest Pair Code – Redundant Self-Comparison and Wrong Pairing Logic

Analysis of the Nested Loop Structure

The given code:

```
python
import math
def closest_pair(points):
    min_dist = float('inf')
    for i in range(len(points)):
        for j in range(len(points)):
            dist = math.sqrt((points[i][0]-points[j][0])**2 +
                             (points[i][1]-points[j][1])**2)
            if dist < min_dist:
                min_dist = dist
    return min_dist
```

Both loops iterate over the **same range**, `range(len(points))`. This means for every value of `i`, the inner loop's `j` also takes the value `j = i` at some point — this is the **self-comparison** problem.

Why Self-Comparison Breaks Correctness

1. **When $i == j$:** The code compares a point with itself:
 $\text{dist} = \sqrt{(x_i - x_i)^2 + (y_i - y_i)^2} = \sqrt{0} = 0$
2. **Since distance 0 is the smallest possible value**, the condition `dist < min_dist` becomes true almost immediately — as early as the very first iteration (`i=0, j=0`). Once this happens, `min_dist` gets set to **0** and can never be replaced by anything smaller (since 0 is the theoretical minimum for any distance).
3. **Result:** The function **always returns 0**, no matter what points are given as input. This is a critical **logical bug**, since it completely defeats the purpose of finding the closest pair among *distinct* points.
4. **Redundant computation (even ignoring the bug):**
 - The loop performs **n^2 total comparisons**, of which:
 - **n comparisons are self-comparisons** (`i == j`) — completely wasted, since they always yield 0.
 - Each **distinct pair (i, j)** is computed **twice**: once as (`i, j`) and once as (`j, i`) — since distance is symmetric (`distance(i,j) = distance(j,i)`), this duplicates half of all useful work.

Debugged Logic

To fix the algorithm, two conditions must be enforced:

- **$i \neq j$** — eliminate self-comparison.
- **Each unordered pair counted only once** — achieved by restricting the inner loop so `j` only takes values **greater than i** (i.e., `j > i`), instead of starting from 0.

This is done by changing the inner loop range from `range(len(points))` to `range(i + 1, len(points))`.

Corrected and Optimized Code

```
python
import math
```

```
def closest_pair(points):
```

```
    """
```

```
    Brute-force Closest Pair of Points algorithm.
```

```
    Fixes applied:
```

- ```
 - Avoids self-comparison ($i == j$), which previously caused
 the function to always return 0.
 - Avoids duplicate comparisons: each unordered pair (i, j)
 is evaluated exactly once instead of twice.
```

```
 Parameters:
```

```
 points: list of tuples/lists, e.g. [(x1, y1), (x2, y2), ...]
```

```
 Returns:
```

```
 min_dist: the smallest Euclidean distance between any
 two DISTINCT points in the list.
```

```
 """
```

```
 n = len(points)
```

```
Edge case: need at least 2 points to form a valid pair
if n < 2:
 return float('inf') # No valid pair exists

min_dist = float('inf')
closest_points = None # Optional: store the actual closest pair

Outer loop: fixes the first point of the pair
for i in range(n):
 # Inner loop starts at i + 1:
 # -> ensures j != i (no self-comparison)
 # -> ensures each pair (i, j) with j > i is checked only ONCE
 for j in range(i + 1, n):
 dist = math.sqrt((points[i][0] - points[j][0]) ** 2 +
 (points[i][1] - points[j][1]) ** 2)

 if dist < min_dist:
 min_dist = dist
 closest_points = (points[i], points[j]) # optional tracking

 return min_dist # can also return closest_points if needed
```

### Key Fixes Summarized

| Issue                     | Original (Buggy) Code                                | Corrected Code                                            |
|---------------------------|------------------------------------------------------|-----------------------------------------------------------|
| Self-comparison (i == j)  | Inner loop range(len(points)) includes j = i         | Inner loop range(i+1, n) starts strictly after i          |
| Duplicate pair evaluation | (i,j) and (j,i) both computed separately             | Each unordered pair computed exactly once (j > i only)    |
| Always returns 0          | Caused by distance(i,i) = 0 always being the minimum | Returns the true minimum distance between distinct points |
| No edge-case handling     | Fails silently for < 2 points                        | Explicit check returns inf if fewer than 2 points exist   |

### Time Complexity Analysis of the Corrected Solution

#### Counting comparisons:

For each i, the inner loop runs from i+1 to n-1:

- i = 0 → (n-1) comparisons
- i = 1 → (n-2) comparisons
- ...
- i = n-2 → 1 comparison
- i = n-1 → 0 comparisons

**Total comparisons = (n-1) + (n-2) + ... + 1 + 0 = n(n-1)/2**

This is **exactly half** the  $n^2$  comparisons performed by the buggy version (which included n wasteful self-comparisons and duplicated every distinct pair).

| Aspect                    | Complexity                                                                                              |
|---------------------------|---------------------------------------------------------------------------------------------------------|
| Original (buggy) code     | $O(n^2)$ comparisons — but always logically incorrect (returns 0)                                       |
| Corrected code            | $O(n^2)$ — specifically $n(n-1)/2 \approx n^2/2$ comparisons                                            |
| Asymptotic classification | Still <b><math>O(n^2)</math></b> (the constant factor of $\frac{1}{2}$ does not change the Big-O class) |
| Space complexity          | $O(1)$ additional space (excluding input storage)                                                       |

**Note:** The corrected version remains  $O(n^2)$  asymptotically since brute-force behavior is intentionally preserved, as required by the question. However, it performs **exactly half the raw operations** of the buggy version and — most importantly — is now **functionally correct**. A true asymptotic improvement to  **$O(n \log n)$**  would require the **Divide and Conquer Closest Pair algorithm** (which recursively splits points by x-coordinate and merges results using a narrow strip check), but that is outside the scope of a brute-force solution.

#### Verification of Correctness

**Example:** points = [(0,0), (3,4), (1,1)]

The corrected code evaluates only the **3 distinct pairs**:

- $(0,0) - (3,4)$ : distance =  $\sqrt{9+16} = 5.0$
- $(0,0) - (1,1)$ : distance =  $\sqrt{1+1} \approx 1.414$
- $(3,4) - (1,1)$ : distance =  $\sqrt{4+9} \approx 3.606$

**Minimum = 1.414**, correctly identified between points  $(0,0)$  and  $(1,1)$  — a valid, nonzero result. This confirms the fix works correctly, unlike the original buggy code, which would have returned **0** for any input due to self-comparison.

### Interpretation

This bug is a classic example of **incorrect loop boundary design** in nested-loop algorithms — reusing the same range for both loop variables silently allows invalid  $(i, j)$  pairs to be included. In brute-force geometric algorithms like Closest Pair, correctness depends on precisely defining what constitutes a "valid pair": here, it must be a **pair of distinct, unordered points**. The fix — changing the inner loop to  $\text{range}(i+1, n)$  — is a standard technique whenever a nested loop should consider **combinations rather than permutations**, and as a side benefit, it also **halves the redundant computation**, all while preserving the required  $O(n^2)$  brute-force nature of the algorithm.